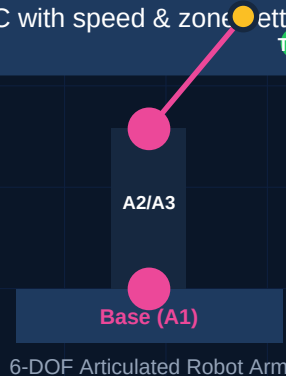


KUKA Robot Programming

KRL Language | KRC4 Controller | PTP / LIN / CIRC | I/O & Subprograms

What You Will Learn

- KUKA robot models: KR 6, KR 16, KR 210 R2700 series
- KRC4 controller architecture and SmartPad teach pendant
- KRL language: variables, data types, operators
- Motion instructions: PTP, LIN, CIRC with speed & zone settings TCP



Awet G. Nway

Founder, AYE Tech Hub | Industrial Robotics & Automation Specialist

ROBOT-005: KUKA Robot Programming

KUKA Robotics, founded in 1898 in Augsburg, Germany, is one of the world's leading manufacturers of industrial robots. KUKA robots are widely used in automotive manufacturing, welding, material handling, palletising, and precision assembly. The KUKA KRC4 controller and KRL programming language are the current industry standard for KUKA systems. This lesson covers the fundamentals of KUKA robot programming that every automation engineer must know.

Learning Objectives

- Identify KUKA robot models and their specifications for application selection
- Navigate the KRC4 controller architecture and SmartPad teach pendant interface
- Declare variables and use data types in the KRL programming language
- Write PTP, LIN, and CIRC motion instructions with speed and zone settings
- Handle digital I/O signals and create subprograms in KRL
- Apply safety practices specific to KUKA robot programming and commissioning

1. KUKA Robot Models — Choosing the Right Robot

KUKA organises its robots by payload capacity (kg) and reach (mm). The correct choice depends on the task's maximum payload, required reach, and workspace constraints. All KUKA robots use the same KRL language and KRC4 controller, so programming skills transfer across the entire range.

Model	Payload	Reach	Axes	Typical Application
KR 6 R700	6 kg	706 mm	6	Small part assembly, dispensing, testing
KR 16 R2010	16 kg	2010 mm	6	Welding, machine tending, handling
KR 60 R2100	60 kg	2100 mm	6	Spot welding, assembly, palletising
KR 120 R2700	120 kg	2700 mm	6	Heavy part handling, press tending
KR 210 R2700	210 kg	2700 mm	6	Automotive body-in-white, heavy pallets
KR 1000 TITAN	1000 kg	3200 mm	6	Foundry, ship building, large press lines

KR Series Robot Axes — The 6 Degrees of Freedom

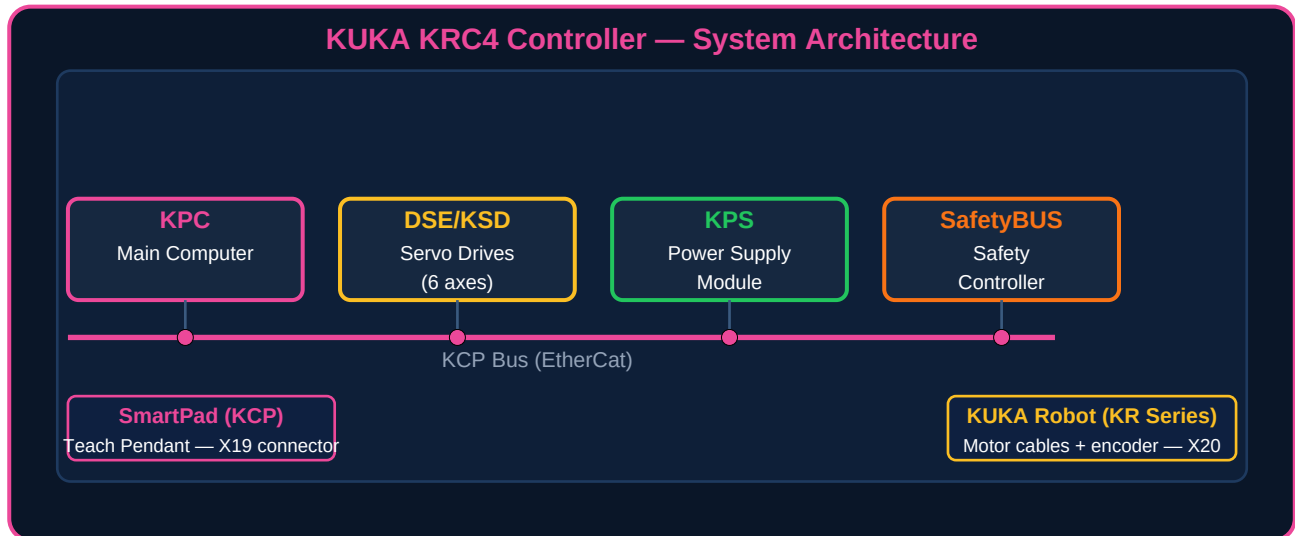
All standard KUKA articulated robots have 6 rotary axes (A1–A6). Each axis has defined motion limits to prevent mechanical overtravel. The robot's position is described by the combination of all six axis angles.

Axis	Joint Name	Motion	Typical Range	Motion Limit if Exceeded
A1	Base rotation	Rotation about vertical axis	±185°	Software limit stop → fault

A2	Shoulder	Forward/backward tilt of upper arm	-135° to $+20^{\circ}$	Hardware mechanical stop
A3	Elbow	Upper arm elevation	-120° to $+155^{\circ}$	Hardware mechanical stop
A4	Wrist rotation	Rotation of forearm about its axis	$\pm 350^{\circ}$	Software limit $\rightarrow$ fault
A5	Wrist bend	Up/down motion of wrist	$\pm 130^{\circ}$	Hardware stop
A6	Flange rotation	Rotation of tool flange	$\pm 350^{\circ}$	Software limit $\rightarrow$ fault

2. KRC4 Controller Architecture

The KRC4 (KUKA Robot Controller 4) is the standard control cabinet for all KUKA robots from 2011 onwards. It integrates the industrial PC (KPC), servo drives (KSD), power supply (KPS), and safety controller into one compact cabinet. Understanding the architecture helps engineers connect the robot to external systems, configure safety, and diagnose faults.



Component	Function	Key Specifications
KPC (KUKA PC)	Runs Windows 10 IoT + KUKA System Software (KSS). Executes KRL programs. Manages HMI.	Intel Core processor, 16 GB SSD, real-time OS extension (VxWorks RTOS core)
KSD (Servo Drives)	One drive module per axis. Converts DC bus voltage to variable AC for servo motors.	6 axes standard; up to 8 axes with external axis. EtherCAT communication to KPC.
KPS (Power Supply)	Rectifies AC mains to DC bus. Supplies all servo drives and control electronics.	3-phase 400V AC input; internal DC bus 600V; integrated braking resistor
SafetyBUS / X11	Connects external E-stop, safety gates, enabling switch to certified safety controller.	PL e / SIL 3 safety category. Monitors all safety inputs with dual-channel logic.
SmartPad (KCP)	Colour touch-screen teach pendant. All robot jogging, programming, and mode selection.	IP54, 8.4" display, 3-position enabling device, emergency stop, mode selector key

SmartPad Key Controls

- **6D mouse (SpaceMouse):** Jogs robot in Cartesian or joint space. Hold the enabling button (middle position) and push the mouse in any direction. T1 mode only.
- **Mode selector:** T1 = Test Manual $\leq 250\text{mm/s}$, T2 = Test Automatic full speed (programmer present), AUT = Automatic (cell running), AUT EXT = Automatic External (PLC start signal).
- **Enabling device (dead-man switch):** Must be held in the middle position for robot motion in T1/T2 modes. Release or squeeze fully → robot stops immediately. Prevents accidental motion.

- **Program keys:** START (play), STOP, RESET, and the step-mode selector (single step / continuous / motion step) on the SmartPad front panel.

3. KRL — KUKA Robot Language

KRL (KUKA Robot Language) is a structured, high-level programming language similar to Pascal. Every KRL program has a .SRC file (executable code) and a matching .DAT file (data declarations for positions, tools, and constants). Programs are created and edited directly on the SmartPad or offline using KUKA.WorkVisual or KUKA.Sim Pro.

KRL Program File Structure — .SRC and .DAT Files

Program File (.SRC)

```
DEF MyProgram()  
; Declarations  
DECL REAL speed  
DECL INT counter  
; Initialization  
speed = 100.0  
; Instructions  
PTP HOME Vel=30%  
LIN P1 CPDAT1  
WAIT SEC 1.5  
END
```

Data File (.DAT)

```
DEFDAT MyProgram  
; Position data  
DECL E6POS P1  
{X 500,Y 0,Z 800}  
DECL E6POS HOME  
{X 0,Y 0,Z 1200}  
; Tool/Base data  
DECL TOOL_DATA  
tool1={X 0,Z 100}  
ENDDAT
```

KRL Data Types

Data Type	Size	Range / Values	KRL Declaration Example	Use Case
BOOL	1 bit	TRUE or FALSE	DECL BOOL gripper_open	Digital output states, flags, conditions
INT	32 bit	−2,147,483,648 to +2,147,483,647	DECL INT part_count	Counters, discrete quantities, loop indices
REAL	32 bit float	±1.1 × 10 ^{−3} to ±3.4 × 10 ³	DECL REAL weld_speed	Speeds, distances, temperatures, analogue values
CHAR	8 bit	Single ASCII character	DECL CHAR key_input	Character storage, menu selections
E6POS	Structure	X, Y, Z (mm) + A, B, C (°) + S, T	DECL E6POS pickup_pos	Cartesian robot positions with orientation
E6AXIS	Structure	A1–A6 axis angles in degrees	DECL E6AXIS home_axis	Pure axis-space positions (no Cartesian)
FRAME	Structure	X, Y, Z, A, B, C (coordinate frame)	DECL FRAME base_frame	Coordinate system definition (tool or base)

KRL Operators and Control Flow

Category	Syntax / Keywords	Example
Assignment	:= (assign value)	speed := 150.0

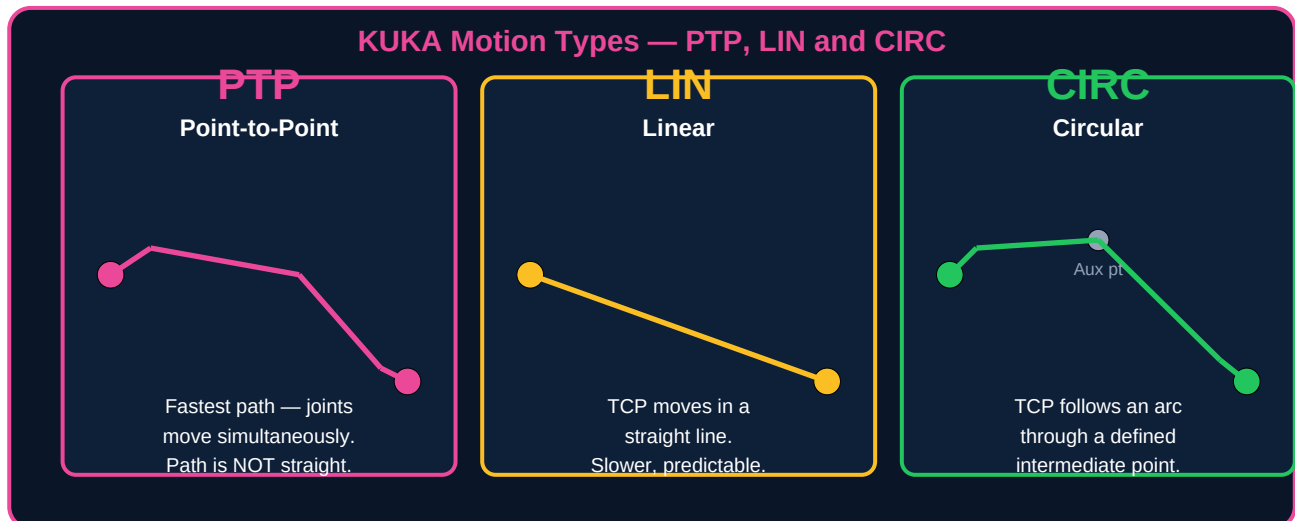
ayetechub.com | Free Engineering Education

Page 6

Arithmetic	+ – * /	result := (a + b) * 2.5
Comparison	== != < > <= >=	IF count >= 10 THEN ...
Logical	AND OR NOT	IF (sensor1 == TRUE) AND (sensor2 == FALSE) THEN
IF / ELSE	IF ... THEN ... ELSE ... ENDIF	IF temp > 80.0 THEN cool_fan := TRUE ENDIF
LOOP / ENDLOOP	Infinite loop with EXIT to break	LOOP ... IF done == TRUE THEN EXIT ENDIF ... ENDLOOP
FOR loop	FOR i = start TO end STEP n	FOR i=1 TO 10 STEP 1; count:=count+1; ENDFOR
WHILE loop	WHILE condition DO ... ENDWHILE	WHILE part_present == FALSE DO WAIT SEC 0.1 ENDWHILE

4. Motion Instructions — PTP, LIN, CIRC

Every robot move in a KRL program uses one of three motion types. Choosing the correct type affects path predictability, cycle time, and collision avoidance. PTP is fastest; LIN and CIRC provide predictable TCP paths required for welding, dispensing, and cutting.



Motion Instruction Syntax

Instruction	Syntax	Key Parameters	When to Use
PTP	PTP target Vel=n% PDAT1	Vel: 1-100% of max joint velocity PDAT: motion parameters block	Moving between non-critical positions. Fastest. Path not guaranteed – avoid obstacles.
PTP HOME	PTP HOME Vel=30% DEFAULT	HOME: built-in home position Always use slow speed on first run	Moving robot to safe home position at program start and end.
LIN	LIN target Vel=n mm/s CPDAT1	Vel: TCP speed in mm/s CPDAT: Cartesian path parameters	Welding seams, dispensing, cutting, assembly – wherever straight-line path is required.
CIRC	CIRC auxpoint target Vel=n mm/s CPDAT2	auxpoint: intermediate point on arc Angle: optional arc override angle	Circular welds, arc dispensing paths, any circular or curved TCP trajectory.
WAIT SEC	WAIT SEC 1.5	Time in seconds (REAL)	Pausing for process dwell (glue cure, part settle, clamp close time).
WAIT FOR	WAIT FOR (\$IN[1] == TRUE)	Any Boolean condition	Synchronising with external signals – wait for sensor, conveyor, PLC handshake.

Approximate Positioning — Zone (C_DIS, C_VEL, C_ORI)

By default, a robot decelerates to zero at each taught point (exact positioning). For continuous paths with smooth blending between moves, KRL provides approximate positioning using Continuous Distance (C_DIS) or Continuous Velocity (C_VEL) settings. The robot begins the next move before fully reaching the current point, creating a rounded corner path.

Approximation Type	Syntax Suffix	Behaviour	Use When
Exact positioning (default)	No suffix	Robot decelerates to zero at every point. Slower cycle time. Precise position guaranteed.	Assembly, pick & place, critical position accuracy required
C_DIS (distance)	LIN P2 Vel=200 CPDAT C_DIS	Blending starts when TCP is within a set distance of the target. Smooth corner.	General path following, conveyor tracking, spray paths
C_VEL (velocity)	LIN P2 Vel=200 CPDAT C_VEL	Blending triggered when TCP velocity matches the next segment velocity. Smoothest transition.	High-speed continuous paths, arc welding, coating
C_ORI (orientation)	PTP P3 Vel=50% PDAT C_ORI	Blends tool orientation smoothly during approach. Position still exact.	Welding with tool angle changes, sealing applications

5. Digital I/O in KRL

KUKA robots communicate with external devices (grippers, clamps, sensors, conveyors, PLCs) via digital I/O. The KRC4 has a built-in I/O bus (KUKA.ProfiNet, KUKA.Profibus, or KUKA.EtherNet/IP) with up to 4096 digital inputs and outputs depending on configuration.

Operation	KRL Syntax	Description
Read digital input	<code>\$IN[1]</code>	Read state of digital input 1. Returns TRUE/FALSE. E.g.: <code>WAIT FOR \$IN[1] == TRUE</code>
Set digital output	<code>\$OUT[1] = TRUE</code>	Set digital output 1 HIGH (24VDC). E.g.: <code>\$OUT[1] = TRUE</code> to open gripper
Clear digital output	<code>\$OUT[1] = FALSE</code>	Set digital output 1 LOW (0V). E.g.: <code>\$OUT[1] = FALSE</code> to close gripper
Pulse output	<code>PULSE(\$OUT[5], TRUE, 0.3)</code>	Set <code>\$OUT[5] = TRUE</code> for 0.3 seconds then automatically clear it
Read analogue input	<code>\$ANIN[1]</code>	Read analogue input 1 as REAL (–1.0 to +1.0 normalised). Multiply by max range.
Set analogue output	<code>\$ANOUT[1] = 0.5</code>	Set analogue output 1 to 50% of max (e.g. 10V if range is 0–20V)
Interrupt on input	<code>INTERRUPT DECL 1 WHEN \$IN[5] DO brake_proc()</code>	Define interrupt: when input 5 triggers, call <code>brake_proc</code> subprogram immediately

6. Subprograms and Functions in KRL

KRL supports modular programming through subprograms (DEF/END blocks within the same .SRC file) and global subprograms (separate .SRC files). Subprograms can receive parameters and return values, enabling reusable code for repeated operations.

Type	Syntax	Use Case
Local subprogram	<code>DEF pick_part() ... END</code>	Called within the same program. Defined after the main END. No parameters by default.
Subprogram with params	<code>DEF move_xyz(REAL x, REAL y, REAL z) ... END</code>	Passes values in. Robot moves to X, Y, Z position calculated from parameters.
Function (returns value)	<code>DEFFCT REAL calc_distance(REAL a, REAL b) RETURN SQRT(a*a + b*b) ENDFCT</code>	Returns a REAL result. Used in expressions: <code>dist = calc_distance(dx, dy)</code>
Global subprogram	Separate .SRC file: <code>MyGlobal.src</code>	Available to all programs in the robot. Called by name: <code>MyGlobal()</code> . Commonly used for utility routines.

Calling a subprogram	<code>pick_part()</code> or with params: <code>move_xyz(500.0, 200.0, 300.0)</code>	Simply write the name. For params, provide values in brackets matching the declaration order.
----------------------	--	---

7. KUKA Safety Rules for Programmers

- **Always start in T1 mode at reduced speed (<250 mm/s)** when running a new or modified program for the first time. Only switch to AUT or AUT EXT when the program is verified safe.
- **Test every taught position individually** using single-step mode (step key on SmartPad) before running the full program in continuous mode.
- **Use proper tool and base data.** Programming without correct tool/base data means the robot's calculated path does not match reality — collision risk is high. Verify TOOL_DATA and BASE_DATA values match the physical setup.
- **Define workspace limits.** Use KSS Mastering/Calibration to set axis software limits and Cartesian workspace monitoring zones that match the actual cell dimensions.
- **Never bypass the enabling device.** The 3-position enabling switch must always be used in T1/T2. Taping or clamping it defeats the purpose and is a serious safety violation.

8. Practice Exercises

Practice these exercises using KUKA.Sim Pro (offline simulation), KUKA KRC4 with real hardware in T1 mode, or the KUKA.SimPro virtual controller available in the AYE Tech Hub robotics resources section.

#	Exercise	Steps
1	Teach and recall HOME position	1. Jog robot to a safe home position using SmartPad. 2. Teach the position as HOME in the .DAT file. 3. Write a one-rung program: PTP HOME Vel=30% DEFAULT and run in T1 mode.
2	Teach 4 positions and run a cycle	1. Jog and teach P1 (approach), P2 (pick), P3 (lift), P4 (place). 2. Write a program with PTP moves between all positions. 3. Verify correct path in T1 single-step mode.
3	LIN straight-line move	1. Teach two positions 300 mm apart on the same height. 2. Program a LIN move at 200 mm/s. 3. Observe that the TCP travels in a straight line — compare with PTP between same points.
4	Gripper I/O control	1. Set \$OUT[1] = TRUE at pick position (gripper closes). 2. Add WAIT SEC 0.5 for gripper close time. 3. Set \$OUT[1] = FALSE at place position (gripper opens).
5	Wait for sensor input	1. Simulate a sensor on \$IN[1]. 2. Add WAIT FOR \$IN[1]==TRUE before the pick move. 3. Run the program and observe robot waits until input is set.

Lesson Summary

Topic	Key Takeaway
KUKA Robot Models	Payload 6–1000 kg. All 6-axis articulated arms. Same KRL language and KRC4 controller across all models.
KRC4 Architecture	KPC (PC) + KSD (servo drives) + KPS (power) + SafetyBUS. SmartPad connects via X19 (6m cable standard).
SmartPad Modes	T1 ≤250mm/s (programming) → T2 full speed (verify) → AUT (auto run) → AUT EXT (PLC control).
KRL Data Types	BOOL, INT, REAL, CHAR, E6POS (Cartesian position), E6AXIS (joint position), FRAME (coordinate system).
Motion Types	PTP = fastest, non-straight path. LIN = straight TCP path. CIRC = arc through intermediate point.
Approximate Positioning	C_DIS / C_VEL suffix blends moves for smooth continuous paths. Default = exact positioning at every point.
Digital I/O	\$IN[n] reads inputs. \$OUT[n]=TRUE/FALSE sets outputs. PULSE for timed pulses. WAIT FOR for synchronisation.
Subprograms	Local DEF/END subprograms. Parametric subprograms. DEFFCT returns values. Global programs in separate .SRC files.

What Is Next — ROBOT-006

ROBOT-006: Fanuc Robot Systems — Fanuc R-2000iC and M-20iA robot series, R-30iB Plus controller architecture, FANUC TP (Teach Pendant) programming, TP motion instructions (J, L, C), position registers, I/O setup, and macro programs. Comparison of KUKA KRL vs FANUC TP vs ABB RAPID programming approaches.

Resource	Details
PDF Series	ROBOT-001 through ROBOT-030 at ayetechub.com/pdfs.html
KUKA Sim Pro	Free 30-day trial — download from kuka.com/en/services/kuka-college/software-downloads
Community	Ask questions in the AYE Tech Hub Telegram group: t.me/ayetechub